Máster en Desarrollo con Inteligencia Artificial · Big School

Trabajo Fin de Máster · Accesibilidad 360

Anexo C. Especificaciones del proyecto

Autor: Jorge Serna
Tutor: Brais Moure

Índice

Anexo C. Especificaciones del proyecto....3
Especificación 000: Especificación del producto 4
Especificación 001: Modelo de dominio 11
Especificación 002: Arquitectura del sistema 25
Especificación 003: Infraestructura del proyecto....34
Especificación 010: Autenticación y autorización 40
Especificación 020: Gestión de usuarios (roles y autorización)....49
Especificación 030: Gestión de establecimientos (1.ª entrega: creación)....55
Especificación 040: Valoraciones de accesibilidad (1.ª entrega: crear y mostrar) 62
Especificación 050: Fotografías de establecimientos (1.ª entrega: subida única) 68
Especificación 060: Geocodificación y mapa del establecimiento (1.ª entrega) 72
Especificación 070: Búsqueda y filtros de establecimientos (1.ª entrega)....76
Especificación 080: Edición y eliminación de establecimientos (1.ª entrega)....79
Especificación 090: Mapa global de establecimientos (1.ª entrega) 82
Especificación 100: Panel principal (1.ª entrega)....85

Anexo C. Especificaciones del proyecto

Este anexo reproduce las catorce especificaciones que guiaron el desarrollo, en el orden de su numeración: cuatro de núcleo (000 a 003) y diez de funcionalidades (010 a 100). Se incluyen tal como se redactaron durante el trabajo, sin ediciones de contenido, para que sea posible contrastar lo que se pidió con lo que se construyó.

Las especificaciones son documentos de trabajo y no una descripción final del producto. Pueden contener decisiones anteriores a su implementación, funciones previstas que no llegaron a incluirse, un estado de aprobación que no se actualizó y referencias a otras especificaciones. **Cuando una especificación difiere del comportamiento de la versión entregada, prevalece el código, que es la fuente de verdad, y su descripción está en los capítulos 3 a 6 de la memoria.** Los diagramas figuran en su notación de origen (Mermaid) y no como imágenes.

Especificación 000: Especificación del producto

Archivo: core/000-product-specification.md

Especificación del producto

Proyecto: Accesibilidad 360

Versión: 1.0

Estado: Aprobada

1. Propósito

Accesibilidad 360 es una plataforma web colaborativa que permite consultar y compartir información sobre la accesibilidad de establecimientos públicos y privados.

Su objetivo es facilitar la toma de decisiones de personas con discapacidad o movilidad reducida mediante información aportada por la comunidad.

La plataforma no pretende sustituir certificaciones oficiales de accesibilidad, sino complementar la información disponible mediante experiencias y evaluaciones reales de los usuarios.

2. Problema

Actualmente es difícil conocer con antelación si un establecimiento es realmente accesible.

Aunque algunos servicios muestran información básica, normalmente:

- está incompleta;
- no está actualizada;
- no incluye fotografías;
- no permite valorar distintos aspectos de la accesibilidad;
- depende únicamente de información proporcionada por el propietario.

Esto obliga a muchas personas a desplazarse sin saber si podrán acceder al establecimiento.

3. Objetivos del producto

El producto debe permitir:

- localizar establecimientos;
- consultar su nivel de accesibilidad;
- registrar nuevos establecimientos;
- compartir evaluaciones objetivas;
- subir fotografías;
- consultar un mapa interactivo;
- buscar establecimientos mediante filtros.

4. Público objetivo

Usuarios principales:

- Personas usuarias de silla de ruedas.
- Personas con movilidad reducida.
- Personas con discapacidad física.
- Personas mayores.
- Familiares y acompañantes.

Usuarios secundarios:

- Asociaciones.
- Ayuntamientos.
- Comercios.
- Técnicos de accesibilidad.

5. Actores del sistema

Visitante

Puede:

- navegar por la plataforma;
- consultar establecimientos;
- utilizar el buscador;
- visualizar el mapa.

No puede modificar información.

Usuario registrado

Además puede:

- crear establecimientos;
- valorar establecimientos;
- subir fotografías;
- editar sus propios contenidos;
- gestionar su perfil.

Moderador

Además puede:

- aprobar establecimientos;
- ocultar contenido inapropiado;
- moderar fotografías;
- revisar valoraciones.

Administrador

Control total del sistema.

Puede gestionar:

- usuarios;
- roles;
- categorías;
- contenido;
- configuración.

6. Entidades principales

El sistema estará compuesto inicialmente por las siguientes entidades:

- Usuario
- Establecimiento
- Valoración
- Fotografía
- Categoría
- Municipio
- Provincia

Estas entidades podrán ampliarse en futuras versiones.

7. Flujo principal

El flujo principal del producto será:

1. Un visitante accede a la plataforma.
2. Busca un establecimiento.
3. Consulta su información.
4. Consulta fotografías.
5. Consulta valoraciones.
6. Si desea colaborar, crea una cuenta.
7. Añade un nuevo establecimiento o realiza una valoración.

Este flujo constituye el caso de uso principal del MVP.

8. Funcionalidades del MVP

Gestión de usuarios

- Registro.
- Inicio de sesión.
- Recuperación de contraseña.
- Gestión del perfil.

Gestión de establecimientos

- Crear.

- Editar.
- Consultar.
- Buscar.
- Filtrar.

Valoraciones

Los usuarios podrán valorar diferentes aspectos de accesibilidad.

Cada valoración estará asociada a un establecimiento.

Fotografías

Cada establecimiento podrá contener varias fotografías.

Las imágenes servirán para complementar la información de accesibilidad.

Mapa

Visualización de establecimientos mediante un mapa interactivo.

Administración

Panel básico para moderación y gestión del contenido.

9. Funcionalidades fuera del MVP

No forman parte del alcance inicial:

- aplicación móvil;
- notificaciones push;
- chat;
- mensajería;
- gamificación;
- API pública;
- multidioma;
- integración con redes sociales.

La arquitectura deberá permitir incorporarlas posteriormente.

10. Requisitos no funcionales

El producto deberá ser:

- responsive;
- accesible;
- mantenible;
- seguro;
- fácilmente desplegable;
- fácilmente testeable.

11. Accesibilidad

La propia aplicación deberá cumplir buenas prácticas WCAG.

El objetivo es que cualquier persona pueda utilizarla independientemente de sus capacidades.

La accesibilidad no constituye una funcionalidad adicional, sino un requisito transversal del proyecto.

12. Arquitectura funcional

El producto estará dividido en los siguientes dominios:

- Autenticación
- Usuarios
- Establecimientos
- Valoraciones
- Fotografías
- Búsqueda
- Mapa
- Administración

Cada dominio evolucionará de forma independiente.

13. Restricciones

El producto:

- no utilizará inteligencia artificial;
- no dependerá de servicios de pago;
- deberá desplegarse automáticamente desde GitHub;
- deberá ejecutarse correctamente en Vercel;
- deberá poder reproducirse fácilmente por cualquier evaluador.

14. Criterios de éxito

El proyecto se considerará satisfactorio cuando:

- el MVP esté completamente operativo;
- todas las funcionalidades críticas dispongan de pruebas automatizadas;
- el despliegue sea automático mediante GitHub y Vercel;
- la documentación permita a cualquier desarrollador comprender el proyecto;
- el código sea mantenible y coherente con la arquitectura definida.

15. Evolución prevista

El diseño del sistema deberá facilitar futuras ampliaciones, entre ellas:

- aplicación móvil;
- API pública;
- colaboración con administraciones;
- validación comunitaria de establecimientos;
- nuevas categorías de accesibilidad;
- internacionalización.

Estas funcionalidades no deberán condicionar el desarrollo del MVP.

Especificación 001: Modelo de dominio

Archivo: core/001-domain-model.md

Especificación

ID: SPEC-001

Nombre: Modelo de dominio

Versión: 1.1

Estado: Aprobada

Dependencias:

- 000-product-specification.md

Bloquea:

- 002-system-architecture.md
- 003-infrastructure.md
- 010-authentication-and-authorization.md
- 020-user-management.md
- 030-establishment-management.md
- 035-normalización-geográfica
- 040-accessibility-reviews.md
- 050-photo-management.md

1. Objetivo

Definir el modelo de dominio de Accesibilidad 360.

Este documento constituye la fuente de verdad para todas las entidades del sistema, sus relaciones, restricciones y reglas de negocio.

No contiene decisiones de implementación.

No describe tecnologías.

No define interfaces de usuario.

Las entidades técnicas de autenticación se describen aparte, en la sección 4, únicamente para delimitar qué no pertenece al dominio.

2. Principios

El modelo deberá:

- representar únicamente conceptos del negocio;
- ser independiente del framework utilizado;
- ser fácilmente ampliable;
- evitar duplicidad de información;
- mantener relaciones claras entre entidades.

3. Entidades de dominio

User

Representa un usuario registrado en la plataforma.

Responsabilidades:

- autenticarse;
- mantener un perfil;
- crear establecimientos;
- realizar valoraciones;
- subir fotografías.

Establishment

Representa un establecimiento físico.

Puede corresponder a:

- restaurante;
- bar;
- cafetería;
- comercio;
- supermercado;

- hotel;
- centro sanitario;
- centro público;
- instalación deportiva;
- etc.

Un establecimiento tiene obligatoriamente:

- una categoría;
- una provincia;
- un municipio;
- un creador (propietario inicial).

Un establecimiento puede tener:

- varias fotografías;
- varias valoraciones.

Su ubicación geográfica (latitud y longitud) puede no estar disponible.

AccessibilityReview

Representa una evaluación realizada por un usuario.

Una valoración siempre pertenece a:

- un usuario;
- un establecimiento.

Cada usuario únicamente podrá tener una valoración por establecimiento.

Photo

Representa una fotografía.

Una fotografía pertenece a:

- un establecimiento;
- un usuario.

El archivo vive en almacenamiento externo; en el dominio solo existen su URL y su identificador remoto.

Category

Clasifica los establecimientos.

Categorías iniciales:

- Restaurante
- Bar
- Cafetería
- Comercio
- Supermercado
- Hotel
- Centro sanitario
- Centro público
- Instalación deportiva
- Otro

Province

Provincia española con su código oficial del INE (2 dígitos).

Ejemplos: Madrid (28), Barcelona (08), Sevilla (41).

Municipality

Municipio español con su código oficial del INE (5 dígitos).

Pertenece siempre a una provincia. Hay nombres repetidos entre provincias.

Criterion

Representa un criterio de accesibilidad que puede evaluarse en un establecimiento.

Criterios iniciales, en orden de presentación:

1. Acceso sin escalones
2. Puerta accesible

3. Anchura de paso
4. Espacio de giro
5. Aseo adaptado
6. Ascensor accesible
7. Aparcamiento para personas con movilidad reducida
8. Señalización accesible

Cada criterio podrá reutilizarse en cualquier establecimiento.

Solo algunos criterios admiten la calificación «No aplicable».

CriterionScore

Representa la puntuación que un usuario asigna a un criterio concreto dentro de una valoración.

Cada registro pertenece a:

- una valoración;
- un criterio.

La puntuación es un entero de 0 a 5, o nula cuando el criterio no resulta aplicable al establecimiento. Lo nulo nunca es un cero: los criterios no aplicables se excluyen de las medias.

4. Entidades técnicas (Auth.js)

No pertenecen al dominio. Existen únicamente para la autenticación y se detallan en SPEC-010, no aquí:

- `Account`: vincula un usuario con un proveedor de autenticación.
- `Session`: sesión persistente (reservada; las sesiones actuales son JWT sin persistencia).
- `VerificationToken`: tokens de un solo uso (p. ej. recuperación).

5. Relaciones

User

1:N

Establishment (como creador, obligatorio)

User

1:N

AccessibilityReview

User

1:N

Photo

Category

1:N

Establishment

Province

1:N

Municipality

Province

1:N

Establishment

Municipality

1:N

Establishment

Establishment

1:N

AccessibilityReview

Establishment

1:N

Photo

AccessibilityReview

1:N

CriterionScore

Criterion

1:N

CriterionScore

6. Atributos

User

- id
- name
- email (único)
- emailVerified (opcional, para futura verificación)
- image (avatar, opcional)
- passwordHash (opcional; nunca texto plano)
- role (USER por defecto)
- createdAt
- updatedAt

Establishment

- id
- name
- description (opcional)
- address
- postalCode (opcional)
- latitude (opcional; nula si la geocodificación no la obtuvo)
- longitude (opcional; nula si la geocodificación no la obtuvo)
- provinceId
- municipalityId

- categoryId
- createdById
- createdAt
- updatedAt

No existe puntuación persistida: las medias se calculan dinámicamente y nunca se almacenan.

AccessibilityReview

- id
- establishmentId
- userId
- comment (opcional; opinión general del establecimiento)
- createdAt
- updatedAt

Unicidad: un usuario, una valoración por establecimiento.

Photo

- id
- establishmentId
- userId
- url
- publicId (identificador remoto, para futuras operaciones)
- caption (opcional, reservado)
- isPrimary (prepara múltiples futuras)
- createdAt

Category

- id
- name (único)
- icon (opcional, reservado)
- createdAt

Province

- id
- code (código INE de 2 dígitos, único)
- name (denominación oficial, única)

Municipality

- id
- provinceId
- code (código INE de 5 dígitos, único)
- name

Unicidad: nombre único dentro de su provincia.

Criterion

- id
- name (único)
- description (opcional)
- order (orden de presentación)
- allowsNotApplicable (solo algunos criterios)
- createdAt

CriterionScore

- id
- reviewId
- criterionId
- score (entero 0–5, o nulo si «No aplicable»)

Unicidad: un criterio, una puntuación por valoración.

7. Reglas de negocio

Usuarios

Un email debe ser único.

Un usuario puede eliminar su cuenta.

No puede eliminar contenido perteneciente a otros usuarios.

Solo el creador o un administrador puede editar o eliminar un establecimiento.

Establecimientos

El nombre es obligatorio.

La dirección, la provincia y el municipio son obligatorios (normalizados, sin texto libre).

Debe pertenecer a una categoría.

La ubicación geográfica es opcional: sin coordenadas no hay mapa, pero la ficha sigue siendo válida.

Valoraciones

Un usuario únicamente puede tener una valoración por establecimiento.

Una valoración exige puntuar todos los criterios: con número (0–5) o con «No aplicable» donde esté permitido.

No podrá valorarse un establecimiento inexistente.

Fotografías

Las fotografías deberán estar asociadas a un establecimiento.

En esta versión, un establecimiento admite una única fotografía.

Criterios de accesibilidad

Los criterios son comunes para toda la aplicación.

Cada criterio puede utilizarse en miles de valoraciones.

Escala de puntuación

- 0 Muy deficiente
- 1 Deficiente
- 2 Aceptable

- 3 Buena
- 4 Muy buena
- 5 Excelente

Las puntuaciones nulas («No aplicable») nunca entran en las medias.

Ejemplo: 5, 5, 4 y un «No aplicable» dan (5 + 5 + 4) / 3, nunca / 4.

8. Enumeraciones

Role

- USER
- MODERATOR
- ADMIN

Todos los nuevos usuarios reciben USER.

9. Eventos de dominio

El sistema deberá contemplar los siguientes eventos:

UserRegistered

UserUpdated

EstablishmentCreated

EstablishmentUpdated

EstablishmentDeleted

ReviewCreated

ReviewUpdated

PhotoUploaded

PhotoDeleted

Aunque inicialmente no exista un sistema de eventos, estos conceptos forman parte del dominio.

10. Restricciones

No podrán existir:

- usuarios duplicados;
- categorías duplicadas;
- criterios duplicados;
- provincias duplicadas;
- municipios duplicados dentro de su provincia;
- valoraciones duplicadas (mismo usuario y establecimiento);
- puntuaciones duplicadas (misma valoración y criterio);
- fotografías sin establecimiento;
- establecimientos sin categoría, provincia, municipio o creador.

11. Diagrama del dominio

```
erDiagram

USER ||--o{ ESTABLISHMENT : creates
USER ||--o{ ACCESSIBILITY_REVIEW : writes
USER ||--o{ PHOTO : uploads

CATEGORY ||--o{ ESTABLISHMENT : classifies

PROVINCE ||--o{ MUNICIPALITY : contains
PROVINCE ||--o{ ESTABLISHMENT : locates

MUNICIPALITY ||--o{ ESTABLISHMENT : locates

ESTABLISHMENT ||--o{ ACCESSIBILITY_REVIEW : receives
ESTABLISHMENT ||--o{ PHOTO : contains

ACCESSIBILITY_REVIEW ||--o{ CRITERION_SCORE : contains

CRITERION ||--o{ CRITERION_SCORE : evaluates

USER ||--o{ ACCOUNT : links
USER ||--o{ SESSION : holds
```

12. Glosario

Establecimiento

Lugar físico susceptible de ser visitado por usuarios.

Valoración

Opinión estructurada sobre la accesibilidad de un establecimiento.

Accesibilidad

Grado en que un establecimiento puede ser utilizado por personas con discapacidad o movilidad reducida.

Provincia

División territorial oficial (código INE) a la que pertenece un establecimiento.

Municipio

Localidad oficial (código INE) donde se sitúa un establecimiento.

Criterio

Aspecto concreto de accesibilidad evaluable, reutilizable en cualquier establecimiento.

Fotografía

Imagen de un establecimiento, almacenada externamente y referenciada por URL.

No aplicable

Calificación de un criterio que, por la naturaleza del establecimiento, no le resulta aplicable. No es un cero y no interviene en las medias.

13. Definition of Ready

El modelo estará listo para implementarse cuando:

- todas las entidades estén definidas;
- todas las relaciones estén documentadas;
- todas las reglas de negocio estén identificadas.

14. Definition of Done

Esta especificación se considerará finalizada cuando:

- permita generar el esquema de base de datos;
- permita generar los tipos TypeScript;
- permita implementar Prisma sin ambigüedades;
- no existan entidades pendientes de definir.

Especificación 002: Arquitectura del sistema

Archivo: core/002-system-architecture.md

Especificación

ID: SPEC-002

Nombre: Arquitectura del sistema

Versión: 1.0

Estado: Aprobada

Dependencias:

- 000-product-specification.md
- 001-domain-model.md

Bloquea:

- 003-infrastructure.md
- Todas las especificaciones funcionales.

1. Objetivo

Definir la arquitectura técnica que deberá seguir todo el proyecto.

Este documento establece la organización del código, el flujo de datos, las responsabilidades de cada capa y las decisiones arquitectónicas que deberán respetarse durante todo el desarrollo.

No contiene detalles de implementación.

2. Objetivos de la arquitectura

La arquitectura deberá ser:

- Modular.
- Escalable.
- Fácil de mantener.
- Fácil de probar.

- Fácil de desplegar.
- Independiente de la interfaz de usuario.
- Independiente de la base de datos siempre que sea posible.

3. Stack tecnológico

Framework
Next.js 15 (App Router)

Lenguaje
TypeScript

Modo estricto obligatorio.

Estilos
Tailwind CSS

Componentes
shadcn/ui

Base de datos
PostgreSQL

ORM
Prisma

Validación
Zod

Formularios
React Hook Form

Autenticación
Auth.js

Mapas
Leaflet + OpenStreetMap

Testing

Vitest

React Testing Library

Playwright

4. Arquitectura general

La aplicación seguirá una arquitectura modular basada en funcionalidades.

Cada módulo será responsable de:

- componentes;
- acciones;
- lógica de negocio;
- validaciones;
- pruebas.

No se utilizará una arquitectura organizada únicamente por tipo de archivo.

5. Organización del proyecto

```
app/
components/
features/
lib/
hooks/
types/
config/
prisma/
tests/
docs/
public/
```

Cada directorio tendrá una responsabilidad claramente definida.

6. Organización por módulos

Cada módulo seguirá esta estructura cuando sea necesario:

```
feature/

components/

actions/

services/

schemas/
```

```
hooks/

types/

utils/

tests/
```

No deberán crearse carpetas vacías.

7. Responsabilidades

app/

Contiene exclusivamente:

- rutas;
- layouts;
- páginas;
- loading;
- error;
- route handlers.

No contendrá lógica de negocio.

features/

Contiene la implementación de cada dominio funcional.

Toda la lógica específica deberá residir aquí.

components/

Componentes reutilizables para toda la aplicación.

lib/

Servicios compartidos.

Ejemplos:

- autenticación;
- base de datos;
- utilidades;
- permisos;

- logger.

prisma/

Esquema de base de datos.

Migraciones.

Seeds.

tests/

Pruebas globales del proyecto.

8. Flujo de datos

Siempre que sea posible:

```
Server Component

↓

Server Action

↓

Service

↓

Prisma

↓

PostgreSQL
```

Evitar peticiones HTTP internas entre componentes del mismo proyecto.

9. Server Components

Los Server Components serán la opción por defecto.

Los Client Components solo se utilizarán cuando exista una necesidad real.

Ejemplos:

- mapas;
- formularios complejos;
- modales;
- drag and drop.

10. Server Actions

Se utilizarán para:

- crear;
- editar;
- eliminar;
- operaciones iniciadas mediante formularios.

11. Route Handlers

Se utilizarán únicamente para:

- APIs públicas;
- integraciones futuras;
- webhooks.

No deberán sustituir a las Server Actions.

12. Acceso a datos

Toda interacción con la base de datos deberá realizarse mediante Prisma.

No se utilizará SQL directo salvo necesidad justificada.

13. Validación

Toda entrada deberá validarse mediante Zod.

Las validaciones del cliente mejoran la experiencia.

Las validaciones del servidor garantizan la seguridad.

Ambas podrán coexistir.

14. Gestión del estado

Se priorizará el estado nativo de React.

No utilizar Redux.

Evitar duplicar estado entre servidor y cliente.

15. Manejo de errores

Los errores deberán:

- registrarse;
- tratarse correctamente;
- no mostrar información técnica al usuario.

16. Seguridad

Toda operación sensible deberá validar:

- autenticación;
- autorización;
- integridad de los datos.

Nunca confiar únicamente en validaciones del cliente.

17. Accesibilidad

Toda interfaz deberá cumplir buenas prácticas WCAG.

La accesibilidad forma parte de la arquitectura del sistema.

No constituye una funcionalidad independiente.

18. Rendimiento

Priorizar las capacidades nativas de Next.js.

No introducir optimizaciones prematuras.

Evitar dependencias innecesarias.

19. Diagrama arquitectónico

```
flowchart TD

A[Usuario]

A --> B[Next.js App Router]

B --> C[Server Components]
```

```
C --> D[Server Actions]

D --> E[Services]

E --> F[Prisma ORM]

F --> G[(PostgreSQL)]

B --> H[Client Components]

H --> C
```

20. Restricciones

No utilizar:

- Redux.
- CSS Modules.
- JavaScript.
- SQL directo.
- APIs internas innecesarias.
- Componentes gigantes.
- Hooks sin reutilización.
- Dependencias no justificadas.

21. Definition of Ready

La arquitectura estará lista para implementarse cuando:

- el dominio esté aprobado;
- las decisiones arquitectónicas estén documentadas;
- las responsabilidades estén claramente definidas.

22. Definition of Done

Esta especificación se considerará finalizada cuando permita:

- crear la estructura del proyecto;
- implementar la infraestructura;
- comenzar el desarrollo de funcionalidades sin ambigüedades.

Especificación 003: Infraestructura del proyecto

Archivo: core/003-infrastructure.md

Especificación

ID: SPEC-003

Nombre: Infraestructura del proyecto

Versión: 1.0

Estado: Aprobada

Dependencias

- 000-product-specification.md
- 001-domain-model.md
- 002-system-architecture.md

Objetivo

Crear la infraestructura inicial del proyecto.

Al finalizar esta especificación deberá existir un proyecto completamente funcional preparado para comenzar el desarrollo del MVP.

No deberá implementarse ninguna funcionalidad del negocio.

Alcance

Esta especificación incluye:

- creación del proyecto;
- configuración del entorno;
- instalación de dependencias;
- estructura de carpetas;
- herramientas de calidad;
- herramientas de testing;
- configuración del repositorio.

No incluye:

- autenticación;
- entidades Prisma;
- lógica de negocio;
- interfaz del producto.

Proyecto

Crear una aplicación Next.js utilizando:

- App Router
- TypeScript
- ESLint
- Tailwind CSS

Dependencias

Instalar únicamente las dependencias definidas en la arquitectura del proyecto.

No instalar librerías adicionales.

Configuración

Configurar:

- TypeScript
- ESLint
- Prettier
- Husky
- lint-staged

Componentes

Instalar y configurar:

- shadcn/ui

No generar todavía componentes personalizados.

Base de datos

Configurar Prisma.

Crear:

- schema.prisma

No definir todavía ninguna entidad.

Configurar la conexión mediante variables de entorno.

Testing

Configurar:

- Vitest
- React Testing Library
- Playwright
- axe-core

Crear un ejemplo mínimo de prueba para verificar que la configuración funciona correctamente.

GitHub

Configurar GitHub Actions para ejecutar:

- instalación;
- lint;
- tests;
- build.

Variables de entorno

Crear:

.env.example

Incluyendo únicamente las variables necesarias para la infraestructura.

No incluir credenciales reales.

Estructura del proyecto

Crear únicamente la estructura necesaria para soportar la infraestructura inicial del proyecto.

No crear carpetas correspondientes a funcionalidades del negocio que aún no se hayan implementado.

Las carpetas de cada módulo funcional se crearán cuando se implemente su especificación correspondiente.

Las carpetas compartidas (por ejemplo, `app`, `components`, `lib`, `prisma`, `tests`, `public` o `docs`) sí deberán existir desde el inicio cuando sean necesarias para la infraestructura.

No crear carpetas vacías sin una finalidad clara.

Scripts

Configurar los scripts mínimos:

- dev
- build
- start
- lint
- lint:fix
- test
- test:watch
- test:e2e
- format
- prepare

README

Crear un README inicial que incluya:

- descripción;

- tecnologías;
- requisitos;
- instalación;
- ejecución;
- estructura del proyecto.

Restricciones

- No crear la estructura interna de módulos funcionales (`features/auth`, `features/users`, `features/establishments`, etc.) hasta que se implemente su especificación correspondiente.

No implementar:

- modelos Prisma;
- autenticación;
- páginas;
- componentes del negocio;
- formularios;
- mapas.

Criterios de aceptación

La especificación se considerará implementada cuando:

- el proyecto compile correctamente;
- `npm run build` finalice sin errores;
- ESLint no reporte errores;
- los tests de ejemplo pasen correctamente;
- Playwright esté configurado;
- Prisma esté configurado;
- shadcn/ui esté instalado;
- Husky funcione correctamente;

- GitHub Actions pueda ejecutarse sin errores.

Antes de ejecutar los tests End-to-End, verificar que no existe ningún proceso utilizando el puerto configurado para el servidor de desarrollo.

La infraestructura deberá ser capaz de ejecutar los tests E2E desde un entorno limpio sin intervención manual.

Definition of Ready

Existen:

- arquitectura;
- dominio;
- especificación del producto.

Definition of Done

El proyecto queda preparado para comenzar el desarrollo de la especificación:

010-authentication-and-authorization.md

No deberá existir ninguna funcionalidad del negocio implementada.

Especificación 010: Autenticación y autorización

Archivo: features/010-authentication-and-authorization.md

Especificación

ID: SPEC-010

Nombre: Autenticación y autorización

Versión: 1.0

Estado: Aprobada

Dependencias

- 000-product-specification.md
- 001-domain-model.md
- 002-system-architecture.md
- 003-infrastructure.md

Objetivo

Implementar el sistema completo de autenticación y autorización de Accesibilidad 360.

El sistema deberá permitir identificar a los usuarios, proteger el acceso a determinadas funcionalidades y gestionar los permisos según el rol asignado.

No se implementarán todavía funcionalidades propias del negocio.

Historias de usuario

HU-010-001

Como visitante quiero crear una cuenta para poder colaborar en la plataforma.

HU-010-002

Como usuario registrado quiero iniciar sesión mediante mi correo electrónico y contraseña.

HU-010-003

Como usuario autenticado quiero cerrar mi sesión.

HU-010-004

Como usuario quiero recuperar mi contraseña en caso de olvidarla.

HU-010-005

Como administrador quiero que cada usuario tenga un rol para controlar los permisos del sistema.

Requisitos funcionales

Registro

El sistema permitirá crear una cuenta utilizando:

- nombre
- correo electrónico
- contraseña
- confirmación de contraseña

El correo deberá ser único.

La contraseña nunca se almacenará en texto plano.

Inicio de sesión

El usuario podrá autenticarse mediante:

- correo electrónico
- contraseña

Si las credenciales son incorrectas se mostrará un mensaje genérico.

No deberá indicarse qué dato es incorrecto.

Cierre de sesión

El usuario podrá cerrar su sesión desde cualquier página autenticada.

Recuperación de contraseña

El usuario podrá solicitar la recuperación indicando su correo electrónico.

Durante el MVP el envío de correo podrá simularse.

La arquitectura deberá permitir incorporar posteriormente un proveedor SMTP.

Roles

Existirán los siguientes roles:

- USER
- MODERATOR
- ADMIN

Todos los nuevos usuarios tendrán inicialmente el rol USER.

Sesión

El sistema deberá mantener la sesión entre peticiones.

Las rutas protegidas deberán comprobar siempre la autenticación.

Requisitos no funcionales

- Contraseñas cifradas mediante algoritmos seguros.
- Validación completa mediante Zod.
- Formularios implementados con React Hook Form.
- Accesibilidad WCAG AA.
- Compatible con dispositivos móviles.
- Sin información sensible en mensajes de error.

Flujo funcional

Registro

Visitante

↓

Formulario

↓

Validación

↓

Creación del usuario

↓

Inicio automático de sesión

↓

Redirección al panel principal

Inicio de sesión

Usuario

↓

Formulario

↓

Validación

↓

Autenticación

↓

Creación de sesión

↓

Redirección

Recuperación

Usuario

↓

Solicitud

↓

Generación de token

↓

Simulación de envío

↓

Formulario de nueva contraseña

↓

Actualización

Autorización

Las siguientes rutas requerirán autenticación:

- perfil
- creación de establecimientos
- edición de establecimientos propios
- creación de valoraciones
- subida de fotografías
- panel de administración

Permisos

USER

Puede:

- gestionar su perfil;
- crear establecimientos;
- crear valoraciones;
- subir fotografías.

MODERATOR

Además puede:

- moderar fotografías;
- ocultar contenido;
- revisar establecimientos.

ADMIN

Acceso completo al sistema.

Casos límite

- Registro con correo existente.
- Contraseña demasiado corta.
- Contraseñas diferentes.
- Usuario inexistente.
- Contraseña incorrecta.
- Sesión expirada.
- Usuario deshabilitado (preparado para futuras versiones).
- Token de recuperación inválido.
- Token expirado.

Criterios de aceptación

- El registro funciona correctamente.
- El inicio de sesión funciona correctamente.
- El cierre de sesión funciona correctamente.
- Los usuarios autenticados mantienen su sesión.
- Las rutas protegidas no son accesibles sin autenticación.
- Los permisos dependen del rol.
- Las validaciones funcionan tanto en cliente como en servidor.

Impacto técnico

Esta especificación podrá modificar únicamente:

- app/
- features/auth/
- lib/auth/
- lib/permissions/
- prisma/
- middleware.ts
- types/

- tests/

No podrá modificar funcionalidades pertenecientes a otros módulos.

Archivos que podrán crearse

- features/auth/components/*
- features/auth/actions/*
- features/auth/services/*
- features/auth/schemas/*
- features/auth/types/*
- features/auth/tests/*
- lib/auth/*
- lib/permissions/*
- middleware.ts

Archivos que NO podrán modificarse

- features/establishments/*
- features/reviews/*
- features/photos/*
- features/map/*
- features/search/*
- features/admin/*
- cualquier especificación de negocio posterior

Tests obligatorios

Unitarios

- Validación de formularios.
- Validación de contraseñas.
- Gestión de permisos.
- Utilidades de autenticación.

Integración

- Registro.
- Login.
- Logout.
- Recuperación de contraseña.

End-to-End

- Registro completo.
- Inicio de sesión.
- Cierre de sesión.
- Acceso denegado a rutas protegidas.
- Acceso permitido según rol.

Definition of Ready

Se considera lista para implementarse cuando:

- la infraestructura esté completamente validada;
- Prisma esté configurado;
- Auth.js esté disponible;
- el modelo de dominio esté aprobado.

Definition of Done

La especificación se considerará finalizada cuando:

- todos los tests pasen correctamente;
- existan pruebas E2E del flujo completo;
- las rutas protegidas funcionen;
- el sistema de roles esté operativo;
- el código cumpla las normas definidas en `.ai`;
- no exista deuda técnica conocida.

Restricciones

No implementar todavía:

- gestión avanzada del perfil;
- edición de datos personales;
- administración de usuarios;
- OAuth (Google, GitHub, Microsoft, etc.);
- autenticación multifactor (MFA);
- verificación de correo electrónico;
- bloqueo de cuentas;
- auditoría de accesos.

Estas funcionalidades podrán incorporarse en futuras especificaciones sin modificar la arquitectura definida.

Especificación 020: Gestión de usuarios (roles y autorización)

Archivo: features/020-user-management.md

Especificación

ID: SPEC-020

Nombre: Gestión de usuarios (roles y autorización)

Versión: 1.0

Estado: Aprobada

Dependencias

- 000-product-specification.md
- 001-domain-model.md
- 002-system-architecture.md
- 003-infrastructure.md
- 010-authentication-and-authorization.md

Objetivo

Definir los roles del sistema y las reglas de autorización que controlan el acceso a las rutas protegidas y a la gestión de contenidos según el rol del usuario autenticado.

La gestión del perfil del usuario autenticado (edición del nombre y cambio de contraseña) pertenece a la SPEC-125 y no forma parte de esta especificación.

Alcance

Implementa únicamente:

- Roles USER, MODERATOR y ADMIN.
- Rol USER por defecto en el registro.
- Propagación del rol en la sesión JWT.
- Guard de autenticación para el grupo `(protected)`.
- Guard de administración para `/admin` y las acciones de categorías.

- Regla de gestión por propietario o administrador para establecimientos, valoraciones y fotografías.
- Etiqueta legible del rol en la interfaz.

Roles

Existen los siguientes roles (`enum Role` en Prisma):

- USER
- MODERATOR
- ADMIN

Todos los nuevos usuarios reciben inicialmente el rol USER (`registerUser` lo fija de forma incondicional; el cliente nunca envía el rol).

El rol MODERATOR existe en el modelo y en las etiquetas de la interfaz, pero no dispone de permisos adicionales implementados: en todas las comprobaciones se comporta como USER.

Permisos

USER

Puede:

- gestionar su perfil (SPEC-125);
- crear establecimientos;
- editar y eliminar sus propios establecimientos;
- crear valoraciones;
- eliminar sus propias valoraciones;
- subir fotografías;
- eliminar sus propias fotografías.

MODERATOR

Sin permisos adicionales implementados respecto a USER.

ADMIN

Acceso completo al sistema. Además puede:

- gestionar establecimientos de cualquier usuario;
- eliminar valoraciones de cualquier usuario;
- eliminar fotografías de cualquier usuario;
- gestionar categorías (crear, renombrar, eliminar);
- acceder al panel de administración.

Autorización

Las siguientes rutas requieren autenticación (grupo `app/(protected)`, verificado en servidor con `requireSession()`; sin sesión redirige a `/login`):

- panel principal
- establecimientos (listado, creación, ficha, edición)
- mapa
- perfil
- panel de administración

La interfaz puede ocultar opciones según el rol (por ejemplo, el enlace a `/admin` solo se muestra a administradores), pero la autorización es siempre responsabilidad del servidor.

Guards

- `requireSession()`: punto único de protección del grupo `(protected)`. Las páginas heredan la protección y no deben reimplementarla.
- `isAdmin(actor)`: verdadero únicamente si `role === "ADMIN"`. Base de las comprobaciones de administración (`/admin`, acciones de categorías).
- `canManageOwnerOrAdmin(actor, ownerId)`: verdadero si el actor es el propietario del recurso o administrador. Misma regla para establecimientos, fotografías y valoraciones; cada servicio la aplica contra el propietario persistido en base de datos.
- `canManageEstablishment(actor, ownerId)`: delega en la regla anterior para establecimientos.

El `middleware.ts` no protege rutas; solo refresca la sesión.

Etiqueta de rol

Los valores USER, MODERATOR y ADMIN son códigos internos. En la interfaz se muestran mediante `roleLabel()`:

- Usuario
- Moderador
- Administrador

Nota sobre el rol MODERATOR

Aunque la SPEC-010 menciona el rol MODERATOR, la implementación actual no define permisos diferenciados para dicho rol. En la práctica, el sistema únicamente aplica reglas específicas para ADMIN; cualquier otro rol se comporta como un usuario estándar (USER). La posible evolución del rol MODERATOR queda fuera del alcance de esta versión.

Seguridad

- El rol viaja en el token JWT y en la sesión (`auth.ts`, callbacks `jwt`/`session`); el cliente nunca lo proporciona.
- Toda operación sensible comprueba permisos en el servidor contra el propietario persistido, nunca contra datos enviados por el cliente.
- La edición y eliminación de establecimientos verifica creador o ADMIN antes de actuar.
- La eliminación de valoraciones y fotografías verifica propietario o ADMIN antes de actuar.
- Las acciones de categorías exigen ADMIN verificado en la Server Action.

Casos límite

- Usuario sin sesión accediendo a una ruta protegida.
- Usuario no administrador accediendo a `/admin` o a acciones de categorías.
- Usuario no propietario intentando editar o eliminar contenido ajeno.
- Rol MODERATOR intentando acciones de administración.

Criterios de aceptación

- El registro asigna siempre el rol USER.
- La sesión expone `id` y `role` del usuario autenticado.
- Las rutas protegidas no son accesibles sin autenticación.
- Solo los administradores acceden a `/admin` y gestionan categorías.
- Solo el creador o un administrador edita o elimina un establecimiento.
- Solo el propietario o un administrador elimina una valoración o fotografía.
- El rol se muestra con su etiqueta legible en la interfaz.

Impacto técnico

Esta especificación podrá modificar únicamente:

- lib/permissions/
- lib/auth/ (propagación del rol en sesión)
- lib/utils/ (etiqueta de rol)
- features/establishments/services/establishment-permissions
- tests/

No podrá modificar funcionalidades pertenecientes a otros módulos.

Tests obligatorios

Unitarios

- `isAdmin` (ADMIN verdadero; USER y MODERATOR falso).
- `canManageOwnerOrAdmin` (propietario, administrador, ajeno).
- `canManageEstablishment` (creador, administrador, ajeno).
- Etiqueta de rol.

Integración

- Guard de administración en `/admin`.
- Acciones de categorías solo para ADMIN.
- Edición y eliminación solo por creador o ADMIN.

- Eliminación de valoraciones y fotografías solo por propietario o ADMIN.

No implementar

- administración de usuarios (listado, cambio de rol, bloqueo);
- edición del perfil (SPEC-125);
- cambio de contraseña (SPEC-125);
- OAuth;
- verificación de correo electrónico;
- auditoría de accesos.

Definition of Ready

Se considera lista para implementarse cuando:

- el modelo de dominio esté aprobado;
- Auth.js esté disponible;
- Prisma esté configurado.

Definition of Done

La especificación se considerará finalizada cuando:

- todos los tests pasen correctamente;
- los guards funcionen según los criterios de aceptación;
- el código cumpla las normas definidas en `.ai`;
- no exista deuda técnica conocida.

Especificación 030: Gestión de establecimientos (1.ª entrega: creación)

Archivo: features/030-establishment-management.md

Especificación

ID: SPEC-030

Nombre: Gestión de establecimientos (1.ª entrega: creación)

Versión: 1.0

Estado: Pendiente de aprobación

Dependencias

- 000-product-specification.md
- 001-domain-model.md
- 002-system-architecture.md
- 003-infrastructure.md
- 010-authentication-and-authorization.md

Objetivo

Implementar la primera entrega del módulo de establecimientos: el flujo de creación de nuevas fichas por usuarios autenticados.

Alcance

Implementa únicamente:

- Modelo Prisma Category.
- Modelo Prisma Establishment.
- Relaciones con User.
- Migración Prisma.
- Página "Nuevo establecimiento".
- Formulario.
- Validación Zod.

- React Hook Form.
- Server Action.
- Servicio.
- Persistencia.
- Redirección al listado tras crear.

Decisiones de modelado (arquitectura MVP)

- `municipality` (en lugar de `city` de 001-domain-model.md).
- `province`.
- `postalCode` opcional (`String?`, no se solicita en el formulario de esta entrega; reservado para futuras fases).
- Category: `id`, `name`, `icon` opcional, `createdAt`.
- No crear tablas Municipality ni Province.
- La especificación 001 deberá actualizarse posteriormente para reflejar esta decisión.

Modelo Category

Campos:

- id
- name
- icon (opcional)
- createdAt

No implementar todavía administración de categorías.

Crear un seed con categorías iniciales:

- Restaurante
- Bar
- Cafetería
- Comercio
- Supermercado

- Hotel
- Centro sanitario
- Centro público
- Instalación deportiva
- Otro

El seed deberá ser idempotente.

Modelo Establishment

Implementar únicamente:

- id
- name
- description
- address
- municipality
- province
- postalCode (opcional, no solicitado en el formulario)
- categoryId
- createdById
- createdAt
- updatedAt

Relaciones:

- User 1:N Establishment
- Category 1:N Establishment

No implementar todavía:

- fotografías
- valoraciones
- puntuación

- coordenadas
- accesibilidad

Formulario

Campos:

- nombre
- categoría
- dirección
- municipio
- provincia
- descripción

Todos obligatorios excepto descripción.

Validaciones

Nombre

- mínimo 3
- máximo 120

Dirección

- obligatoria

Municipio

- obligatorio

Provincia

- obligatoria

Categoría

- obligatoria

Seguridad

Solo usuarios autenticados podrán crear establecimientos.

Obtener siempre el usuario desde la sesión.

Nunca aceptar createdById desde el cliente.

Dashboard

El enlace "Establecimientos" dejará de apuntar al placeholder.

Crear:

/establishments/new

Tras crear correctamente:

redirigir a:

/establishments

Aunque el listado todavía sea provisional.

Listado

Crear una página muy sencilla que muestre:

"Próximamente aparecerá aquí el listado de establecimientos."

pero incluyendo un botón:

"Nueva ficha"

para acceder al formulario.

No implementar

- edición
- eliminación
- fotografías
- mapa
- valoraciones

- filtros
- búsqueda
- paginación

Tests

Crear únicamente:

- esquema Zod
- servicio
- Server Action

No crear E2E.

Definition of Ready

La entrega estará lista para implementarse cuando esta especificación esté aprobada.

Definition of Done

La entrega se considerará finalizada cuando:

- la migración `add_establishments` se aplique sin errores;
- el seed idempotente cree las 10 categorías;
- el formulario valide en cliente y servidor;
- solo usuarios autenticados puedan crear (createdById siempre de sesión);
- tras crear se redirija a /establishments;
- `npm run build`, `npm run lint` y `npm test` pasen sin errores.

Política de eliminación

La eliminación de establecimientos queda limitada al propio registro del establecimiento.

En esta fase del proyecto no se define el comportamiento sobre entidades relacionadas (valoraciones, fotografías u otras que se incorporarán en especificaciones posteriores).

La política definitiva de eliminación (cascada, restricción, borrado lógico o cualquier otra estrategia) se decidirá cuando dichos módulos formen parte del dominio.

Mientras tanto, la implementación no deberá asumir ningún comportamiento de eliminación en cascada.

Entrega 2 — Listado y detalle de establecimientos

Objetivo

Sustituir el placeholder inicial por un listado funcional y una página de detalle.

Alcance

Implementado:

- Listado de establecimientos.
- Ordenación por fecha de creación descendente.
- Tarjetas reutilizables (`EstablishmentCard`).
- Estado vacío.
- Enlace a creación.
- Página de detalle.
- `notFound()` cuando el establecimiento no existe.

No implementado todavía:

- Edición.
- Eliminación.
- Búsqueda.
- Filtros.
- Paginación.
- Valoraciones.
- Fotografías.
- Mapa.

Especificación 040: Valoraciones de accesibilidad (1.ª entrega: crear y mostrar)

Archivo: features/040-accessibility-reviews.md

Especificación

ID: SPEC-040

Nombre: Valoraciones de accesibilidad (1.ª entrega: crear y mostrar)

Versión: 1.0

Estado: Aprobada con precisiones (ver Decisiones de modelado y Formulario)

Dependencias

- 000-product-specification.md
- 001-domain-model.md
- 002-system-architecture.md
- 003-infrastructure.md
- 010-authentication-and-authorization.md
- 030-establishment-management.md

Objetivo

Permitir que un usuario autenticado añada una valoración de accesibilidad a un establecimiento y que las valoraciones existentes se muestren en la ficha.

Alcance

Implementa únicamente:

- Modelo Prisma Criterion.
- Modelo Prisma AccessibilityReview.
- Modelo Prisma CriterionScore.
- Relaciones.
- Seed idempotente de criterios.
- Formulario.

- Server Action.
- Servicio.
- Mostrar las valoraciones existentes en la ficha del establecimiento.

Decisiones de modelado (arquitectura MVP)

Simplifican 001-domain-model.md para la primera entrega:

- `Criterion` en lugar de `AccessibilityCriterion`: `id`, `name` (único, para seed idempotente), `description`, `order` (orden de presentación), `createdAt`. Sin `code`, `weight`, `active`, `icon` ni `applicableByDefault` (fases posteriores).
- `CriterionScore` sin `comment` (el comentario vive en la valoración).
- `score` entero de 0 a 5.
- Unicidad: un usuario solo puede tener una valoración por establecimiento (`@@unique([establishmentId, userId])`); un criterio solo se puntúa una vez por valoración (`@@unique([reviewId, criterionId])`).
- Borrado en cascada: al eliminar una valoración se eliminan sus puntuaciones.
- No crear tablas de medias ni estadísticas.
- La especificación 001 deberá actualizarse posteriormente para reflejar esta decisión.

Modelo Criterion

Campos:

- id
- name (único)
- description
- order
- createdAt

Seed inicial idempotente (por nombre), en este orden:

1. Acceso sin escalones
2. Puerta accesible

3. Anchura de paso
4. Espacio de giro
5. Aseo adaptado
6. Ascensor accesible
7. Aparcamiento PMR
8. Señalización accesible

Modelo AccessibilityReview

Campos:

- id
- establishmentId
- userId
- comment (opcional)
- createdAt
- updatedAt

Un usuario solo podrá crear una valoración por establecimiento.

Modelo CriterionScore

Campos:

- id
- reviewId
- criterionId
- score (entero, mínimo 0, máximo 5)

Relaciones

- User 1:N AccessibilityReview
- Establishment 1:N AccessibilityReview
- AccessibilityReview 1:N CriterionScore

- Criterion 1:N CriterionScore

Migración

Nombre previsto: `add_accessibility_reviews`.

Dada la inestabilidad observada de `migrate dev` contra Neon en este entorno, se aplicará con el flujo equivalente `migrate diff --from-url` + `migrate deploy` si fuese necesario.

Escala de puntuación (aprobada)

Significado accesible de cada valor (de 001-domain-model.md). Debe mostrarse al usuario junto a cada puntuación, nunca números sin contexto:

- 0 — Muy deficiente
- 1 — Deficiente
- 2 — Aceptable
- 3 — Buena
- 4 — Muy buena
- 5 — Excelente

Formulario

Mostrar:

- comentario (opcional)
- lista de criterios (todos los del seed, en orden)

Cada criterio tendrá una puntuación obligatoria de 0 a 5.

No utilizar sliders.

Utilizar radios o botones claramente accesibles (grupo `radiogroup` con leyenda por criterio, navegable por teclado).

Cada opción llevará su significado mediante etiqueta visible o `aria-label` según la escala aprobada (p. ej. "3 — Buena").

Ficha del establecimiento

En `/establishments/[id]`, bajo los datos básicos, mostrar:

- formulario de creación (solo autenticados)
- lista de valoraciones existentes, cada una con comentario, autor, fecha y criterios valorados

No calcular todavía:

- media
- puntuación global
- estadísticas

Seguridad

Solo usuarios autenticados podrán valorar (sesión obligatoria en la Server Action; `userId` siempre de la sesión, nunca del cliente).

Un usuario no podrá valorar dos veces el mismo establecimiento (restricción de unicidad en base de datos + error controlado).

No implementar

- edición
- eliminación
- fotografías
- puntuaciones medias
- filtros

Tests

Crear únicamente:

- esquema Zod (casos válidos, puntuaciones fuera de rango, comentario opcional)
- servicio (creación, duplicado por usuario/establecimiento, puntuación inválida)
- Server Action (sin sesión, entrada inválida, éxito)

Prisma mockeado (sin base de datos en tests). No crear E2E.

Definition of Ready

La entrega estará lista para implementarse cuando esta especificación esté aprobada.

Definition of Done

La entrega se considerará finalizada cuando:

- la migración `add_accessibility_reviews` se aplique sin errores;
- el seed idempotente cree los 8 criterios;
- el formulario valide en cliente y servidor con radios accesibles;
- un usuario no pueda valorar dos veces el mismo establecimiento;
- la ficha muestre las valoraciones con comentario, autor, fecha y criterios;
- `npm run build`, `npm run lint` y `npm test` pasen sin errores.

Especificación 050: Fotografías de establecimientos (1.ª entrega: subida única)

Archivo: features/050-photo-management.md

Especificación

ID: SPEC-050

Nombre: Fotografías de establecimientos (1.ª entrega: subida única)

Versión: 1.0

Estado: Aprobada

Dependencias

- 000-product-specification.md
- 001-domain-model.md
- 002-system-architecture.md
- 003-infrastructure.md
- 010-authentication-and-authorization.md
- 030-establishment-management.md

Objetivo

Permitir que un usuario autenticado suba una fotografía a un establecimiento, almacenada en Cloudinary y mostrada en la ficha.

Alcance

Implementa únicamente:

- Modelo Prisma Photo (preparado para múltiples futuras).
- Migración Prisma.
- Formulario de subida (un archivo).
- Validación Zod (tipo, tamaño, dimensiones).
- Server Action con subida servidor-a-servidor.
- Servicio de persistencia (solo URL + publicId).

- Mostrar la fotografía en la ficha.

Decisiones de modelado (arquitectura MVP)

- `Photo`: `id`, `establishmentId`, `userId`, `url` (`secure_url`), `publicId` (imprescindible para futuras eliminaciones/transformaciones), `caption String?` (reservado, no solicitado), `isPrimary Boolean @default(true)` (soporte futuro de múltiples), `createdAt`.
- Relaciones: `Establishment 1:N Photo` y `User 1:N Photo` (Cascade, revisión pendiente como ADR-002/ADR-003).
- La regla "una fotografía por establecimiento" vive en el servicio (error controlado), no en BD, para no bloquear la futura galería.
- Almacenamiento en Cloudinary según ADR-004 (sustituye al almacenamiento local inicial por el filesystem efímero de Vercel).

Validaciones del archivo

- Formatos: JPG (`image/jpeg`), PNG (`image/png`), WebP (`image/webp`).
- Tamaño máximo: 5 MB.
- Dimensiones mínimas: 800 × 600 px (verificadas con la respuesta de Cloudinary tras la subida; si no se alcanzan, se elimina lo subido y se devuelve error controlado, sin dependencias nuevas).

Formulario

Un único `input[type=file]` (`accept="image/jpeg,image/png,image/webp"`).

Si el establecimiento ya tiene fotografía, el formulario se muestra **deshabilitado** (`fieldset disabled`) con el mensaje: "Este establecimiento ya tiene una fotografía. La gestión de múltiples fotografías llegará en una próxima fase."

Sin drag & drop, sin compresión manual, sin previsualización.

Ficha del establecimiento

Si existe fotografía, mostrarla con `next/image` (requiere `images.remotePatterns` para `res.cloudinary.com`).

Seguridad

Solo usuarios autenticados (`userId` siempre de la sesión). El API secret de Cloudinary nunca sale del servidor (subida en la Server Action).

No implementar

- galerías
- carruseles
- edición
- eliminación
- múltiples fotografías
- compresión manual
- drag & drop

Tests

Crear únicamente:

- esquema Zod (tipo, tamaño)
- servicio (SDK mockeado: éxito, duplicado por establecimiento, dimensiones insuficientes con limpieza)
- Server Action (sin sesión, entrada inválida, éxito)

Prisma mockeado. No crear E2E.

Definition of Done

La entrega se considerará finalizada cuando:

- la migración `add_photos` se aplique sin errores;
- un usuario autenticado pueda subir JPG/PNG/WebP ≤ 5 MB y ≥ 800×600;
- solo se persistan `url` y `publicId` (+ metadatos propios);

- la segunda subida al mismo establecimiento devuelva error controlado;
- la ficha muestre la fotografía;
- `npm run build`, `npm run lint` y `npm test` pasen sin errores.

Especificación 060: Geocodificación y mapa del establecimiento (1.ª entrega)

Archivo: features/060-geocoding.md

Especificación

ID: SPEC-060

Nombre: Geocodificación y mapa del establecimiento (1.ª entrega)

Versión: 1.0

Estado: Aprobada

Dependencias

- 000-product-specification.md
- 001-domain-model.md
- 002-system-architecture.md
- 003-infrastructure.md
- 010-authentication-and-authorization.md
- 030-establishment-management.md

Objetivo

Situar cada establecimiento en el mapa a partir de su dirección y mostrar su ubicación en la ficha mediante un mapa individual.

Alcance

Implementa únicamente:

- Geocodificación de la dirección con Nominatim (OpenStreetMap).
- Campos `latitude` y `longitude` anulables en el modelo Establishment.
- Cálculo de coordenadas al crear; recálculo al editar solo si cambia la ubicación.
- Sección «Ubicación» en la ficha con mapa individual o aviso cuando no hay coordenadas.

Decisiones de modelado (arquitectura MVP)

- `Establishment.latitude` y `Establishment.longitude` (`Float?`, anulables): sin coordenadas el establecimiento existe igualmente y la ficha lo indica.
- La dirección consultada se construye con los nombres oficiales de provincia y municipio verificados en base de datos (SPEC-035), nunca con texto libre del formulario.
- Sin tablas de caché ni colas: la geocodificación es una llamada directa durante la creación o edición.

Geocodificación

Se utiliza el servicio Nominatim (OpenStreetMap) para convertir la dirección en coordenadas.

- La consulta se realiza durante la creación del establecimiento, con la dirección y los nombres oficiales de municipio y provincia.
- Al editar, la consulta solo se repite cuando cambian la dirección, el municipio o la provincia; en caso contrario se preservan las coordenadas existentes.
- Cualquier error deja las coordenadas a `null` sin impedir la creación ni la edición.
- El mapa solo se muestra cuando existen coordenadas válidas.

Ficha del establecimiento

En `/establishments/[id]`, sección «Ubicación»:

- Con coordenadas: mapa individual (`EstablishmentMap`) centrado en el punto con zoom 16 y marcador circular con el nombre en popup.
- Sin coordenadas: tarjeta con el aviso «Ubicación no disponible» y el texto «Todavía no se ha podido situar este establecimiento en el mapa.».

Toda la información del mapa existe también en texto (dirección, municipio, provincia); el mapa es complementario.

Mapa individual

- `EstablishmentMap`: Client Component (Leaflet exige DOM), marcador `CircleMarker` único y popup con el nombre.

- `EstablishmentMapLoader`: carga diferida con `ssr: false` y mensaje «Cargando mapa…».
- `MapFrame`: marco compartido con el mapa global (SPEC-090); teselas de OpenStreetMap con atribución obligatoria y `scrollWheelZoom` desactivado para no secuestrar el scroll.

Seguridad

Solo usuarios autenticados crean y editan establecimientos (`userId` siempre de la sesión). La geocodificación se ejecuta siempre en el servidor; la clave de la consulta no expone ningún secreto.

No implementar

- mapa global (SPEC-090);
- re-geocodificación manual;
- caché de coordenadas;
- edición de coordenadas a mano;
- rutas o distancias.

Tests

Crear únicamente:

- servicio de geocodificación (construcción de la consulta, coordenadas de Nominatim, nulos ante fallo, sin red real mediante `fetch` mockeado);
- servicio (geocodificación con nombres oficiales, persistencia de coordenadas, `null` si falla);
- Server Action (geocodificación mockeada, sin red en tests).

Prisma mockeado (sin base de datos en tests). No crear E2E.

Definition of Ready

La entrega estará lista para implementarse cuando esta especificación esté aprobada.

Definition of Done

La entrega se considerará finalizada cuando:

- la migración `add_establishment_coordinates` se aplique sin errores;
- crear con dirección válida persista las coordenadas;
- crear con fallo de geocodificación persista `null` sin bloquear;
- editar preserve las coordenadas si la ubicación no cambia;
- la ficha muestre el mapa o el aviso según corresponda;
- `npm run build`, `npm run lint` y `npm test` pasen sin errores.

Especificación 070: Búsqueda y filtros de establecimientos (1.ª entrega)

Archivo: features/070-search-and-filters.md

Especificación

ID: SPEC-070

Nombre: Búsqueda y filtros de establecimientos (1.ª entrega)

Versión: 1.0

Estado: Aprobada

Dependencias

- 000-product-specification.md
- 001-domain-model.md
- 002-system-architecture.md
- 003-infrastructure.md
- 010-authentication-and-authorization.md
- 030-establishment-management.md

Objetivo

Permitir localizar establecimientos por nombre, categoría y ubicación, y ordenarlos según distintos criterios, desde la página del listado.

Alcance

Implementa únicamente:

- Búsqueda por nombre.
- Filtros por categoría, provincia y municipio.
- Ordenación (recientes, antiguos, nombre A–Z, nombre Z–A, mejor valorados).
- Filtros reflejados en la URL y persistidos como valores iniciales.
- Estado vacío con acciones contextuales.

Decisiones de diseño (arquitectura MVP)

- Un único constructor del `where`: solo las condiciones con valor participan; los valores vacíos se ignoran y un orden desconocido equivale al orden por defecto.
- La ordenación por «Mejor valorados» se obtiene a partir de la valoración media calculada dinámicamente a partir de las valoraciones existentes.
- El formulario es un GET nativo: los filtros viajan en la URL sin JavaScript ni AJAX, por lo que los resultados son enlazables y el navegador conserva el historial.

Filtros

Combinables entre sí:

- Buscar por nombre (coincidencia parcial, sin distinguir mayúsculas).
- Categoría.
- Provincia.
- Municipio (solo habilitado cuando hay provincia elegida; muestra únicamente los municipios de esa provincia).
- Ordenar por: Más recientes (por defecto), Más antiguos, Nombre A–Z, Nombre Z–A, Mejor valorados (por media y, en caso de empate, por fecha).

Listado

En `/establishments`, bajo el título y el acceso a «Nueva ficha», el formulario ofrece «Aplicar filtros» y «Limpiar» (vuelve al listado sin filtros).

- Sin resultados con filtros activos: «No se han encontrado establecimientos con esos filtros.» con acción «Limpiar filtros».
- Sin establecimientos ni filtros: invitación a crear la primera ficha con acción «Nueva ficha».

Seguridad

Solo usuarios autenticados acceden al listado (grupo `(protected)`). Los filtros proceden de la URL y se normalizan en servidor; un valor desconocido nunca altera el comportamiento esperado.

No implementar

- paginación;
- filtros por valoración mínima o criterios;
- búsqueda por cercanía;
- guardado de búsquedas.

Tests

Crear únicamente:

- servicio (condiciones combinadas, valores vacíos ignorados, órdenes en base de datos, valoración ordenada por media con desempate por fecha).

Prisma mockeado (sin base de datos en tests). No crear E2E.

Definition of Ready

La entrega estará lista para implementarse cuando esta especificación esté aprobada.

Definition of Done

La entrega se considerará finalizada cuando:

- buscar por nombre devuelva coincidencias parciales;
- cada filtro combine correctamente con los demás;
- los cinco órdenes respondan según lo descrito;
- los filtros viajen en la URL y persistan como valores iniciales;
- «Limpiar» devuelva al listado sin filtros;
- `npm run build`, `npm run lint` y `npm test` pasen sin errores.

Especificación 080: Edición y eliminación de establecimientos (1.ª entrega)

Archivo: features/080-edit-and-delete.md

Especificación

ID: SPEC-080

Nombre: Edición y eliminación de establecimientos (1.ª entrega)

Versión: 1.0

Estado: Aprobada

Dependencias

- 000-product-specification.md
- 001-domain-model.md
- 002-system-architecture.md
- 003-infrastructure.md
- 010-authentication-and-authorization.md
- 020-user-management.md
- 030-establishment-management.md
- 060-geocoding.md

Objetivo

Permitir que el creador de un establecimiento o un administrador edite su ficha o la elimine, reutilizando el formulario y las validaciones de la creación.

Alcance

Implementa únicamente:

- Página de edición con el formulario precargado.
- Actualización con las mismas validaciones de la creación.
- Eliminación con confirmación desde la ficha.
- Permisos de creador o administrador en todos los pasos.

- Eliminación de las fotografías almacenadas en Cloudinary antes de eliminar su registro en la aplicación.

Decisiones de diseño (arquitectura MVP)

- La edición reutiliza el formulario de creación en modo edición, con los valores actuales como iniciales; no existe un formulario separado.
- La ubicación recalcula sus coordenadas únicamente cuando cambia (SPEC-060); las fotografías y valoraciones no se tocan al editar.
- Al eliminar, las fotografías almacenadas en Cloudinary se eliminan antes que su registro en la aplicación; las valoraciones y fotografías asociadas caen por las eliminaciones en cascada documentadas (ADR-002/ADR-003).

Edición

En `/establishments/[id]/edit`:

- Solo el creador o un administrador ven el formulario; cualquier otro caso responde con `notFound()`, sin revelar si el establecimiento existe.
- Tras guardar, se revalidan la ficha y el listado, y se navega a la ficha actualizada.

Eliminación

En la ficha, solo el creador o un administrador ven las acciones «Editar» y «Eliminar».

- «Eliminar» pide confirmación nativa («¿Eliminar este establecimiento? Esta acción no se puede deshacer.») y muestra los errores de forma accesible.
- Tras eliminar, se revalida el listado y se navega a `/establishments`.

Seguridad

Solo el creador o un administrador, verificado en servidor contra el creador persistido y nunca con datos del cliente. Los errores de inexistencia y de permiso son controlados y no filtran información.

No implementar

- edición o eliminación de valoraciones;
- edición o eliminación de fotografías;
- borrado lógico;
- historial de cambios.

Tests

Crear únicamente:

- permisos (creador, administrador, ajeno);
- servicio (actualización, eliminación con limpieza, inexistente, prohibido);
- Server Actions (sin sesión, entrada inválida, éxito, geocodificación mockeada sin red).

Prisma mockeado (sin base de datos en tests). No crear E2E.

Definition of Ready

La entrega estará lista para implementarse cuando esta especificación esté aprobada.

Definition of Done

La entrega se considerará finalizada cuando:

- solo el creador o un administrador accedan a la edición;
- la edición valide igual que la creación y preserve fotografías y valoraciones;
- la eliminación pida confirmación y elimine las fotografías almacenadas en Cloudinary antes que su registro;
- los permisos se verifiquen en servidor en todos los pasos;
- `npm run build`, `npm run lint` y `npm test` pasen sin errores.

Especificación 090: Mapa global de establecimientos (1.ª entrega)

Archivo: features/090-global-map.md

Especificación

ID: SPEC-090

Nombre: Mapa global de establecimientos (1.ª entrega)

Versión: 1.0

Estado: Aprobada

Dependencias

- 000-product-specification.md
- 001-domain-model.md
- 002-system-architecture.md
- 003-infrastructure.md
- 010-authentication-and-authorization.md
- 020-user-management.md
- 030-establishment-management.md
- 060-geocoding.md

Objetivo

Mostrar todos los establecimientos geolocalizados en un único mapa colaborativo, con acceso a sus fichas desde cada marcador.

Alcance

Implementa únicamente:

- Página `/map` con el mapa global.
- Marcadores por establecimiento con popup enriquecido.
- Resumen textual del número de establecimientos.
- Estado vacío cuando no hay ubicaciones.

Decisiones de diseño (arquitectura MVP)

- Una única lectura con la primera fotografía y las puntuaciones para la media; solo establecimientos con coordenadas.
- El mapa es complementario al resumen textual: la página informa sin depender únicamente del mapa.
- El mapa se carga de forma diferida para garantizar su correcto funcionamiento en el entorno de ejecución de la aplicación, con mensaje de carga.

Mapa global

En `/map`:

- Solo aparecen establecimientos con coordenadas; sin ninguno se muestra «Todavía no hay establecimientos con ubicación en el mapa.».
- Encima del mapa se indica el número: «1 establecimiento en el mapa.» o «N establecimientos en el mapa.».
- La vista se ajusta automáticamente para encuadrar todos los puntos.
- Cada establecimiento tiene un marcador circular coherente con el de la ficha, sin assets adicionales.

El popup de cada marcador muestra el nombre enlazado, la categoría y el municipio, la puntuación media con su recuento, la fotografía principal cuando existe y el enlace «Ver ficha», navegable por teclado.

Se reutiliza el marco compartido con el mapa de la ficha (SPEC-060): teselas de OpenStreetMap con atribución obligatoria y sin secuestrar el scroll.

Seguridad

Solo usuarios autenticados acceden al mapa (grupo `(protected)`). El mapa no expone ningún dato que la ficha no muestre ya.

No implementar

- filtros sobre el mapa;
- rutas o distancias;
- geolocalización del usuario;

- capas o leyendas.

Tests

Crear únicamente:

- servicio (filtrado sin coordenadas, primera fotografía, media y recuento, lista vacía).

Prisma mockeado (sin base de datos en tests). No crear E2E.

Definition of Ready

La entrega estará lista para implementarse cuando esta especificación esté aprobada.

Definition of Done

La entrega se considerará finalizada cuando:

- solo los establecimientos con coordenadas aparezcan en el mapa;
- el resumen textual indique el número exacto;
- cada popup enlace a su ficha con fotografía y puntuación;
- la vista encuadre todos los puntos;
- sin ubicaciones se muestre el estado vacío;
- `npm run build`, `npm run lint` y `npm test` pasen sin errores.

Especificación 100: Panel principal (1.ª entrega)

Archivo: features/100-dashboard.md

Especificación

ID: SPEC-100

Nombre: Panel principal (1.ª entrega)

Versión: 1.0

Estado: Aprobada

Dependencias

- 000-product-specification.md
- 001-domain-model.md
- 002-system-architecture.md
- 003-infrastructure.md
- 010-authentication-and-authorization.md
- 020-user-management.md
- 030-establishment-management.md
- 040-accessibility-reviews.md
- 050-photo-management.md

Objetivo

Ofrecer un resumen de la actividad de la plataforma en la página `/dashboard`: totales, últimos establecimientos, mejor valorados y accesos rápidos.

Alcance

Implementa únicamente:

- Saludo con el nombre del usuario autenticado.
- Cuatro estadísticas totales.
- Bloque protagonista con los 6 últimos establecimientos.
- Ranking con los 6 mejor valorados.

- Cuatro accesos rápidos.
- Estados vacíos.

Decisiones de diseño (arquitectura MVP)

- La página no calcula ni transforma nada: recibe el DTO completamente preparado por el servicio.
- La tarjeta de estadística es un componente compartido (`components/stats`), reutilizado también por el perfil (SPEC-125).
- La arquitectura del módulo está preparada para futuras ampliaciones del panel sin alterar la organización actual.
- Sin gráficos todavía.

Panel principal

En `/dashboard`, bajo el título «Panel principal» y el saludo «Bienvenido, {nombre}» (o el correo si no hay nombre):

- Estadísticas: Establecimientos, Usuarios, Valoraciones y Fotografías, cada una con su total.
- Últimos establecimientos: hasta 6, los más recientes, en tarjetas compactas con fotografía, nombre, categoría, municipio y provincia, estrellas, puntuación media y número de valoraciones. Sin establecimientos: «Aún no hay establecimientos.».
- Mejor valorados: hasta 6, ordenados por valoración media y, en caso de empate, por número de valoraciones y fecha. Los tres primeros se distinguen con medalla (🥇 🥈 🥉) y el primero queda discretamente destacado; el resto muestra su posición numérica. Cada entrada enlaza a su ficha con categoría, media y recuento. Sin valoraciones: «Aún no hay valoraciones.».
- Accesos rápidos: «Nuevo establecimiento» (`/establishments/new`), «Buscar establecimientos» (`/establishments`), «Mapa» (`/map`) y «Mi perfil» (`/profile`).

Seguridad

Solo usuarios autenticados acceden al panel (grupo `(protected)`). El panel no expone ningún dato que las fichas no muestren ya.

No implementar

- gráficos;
- bloque de últimas valoraciones en la página;
- filtros o paginación del panel;
- estadísticas avanzadas.

Tests

Crear únicamente:

- servicio (estadísticas, últimos, ranking, últimas valoraciones, vacíos);
- tarjeta compacta de valoración (enlace, autor, estrellas con media exacta, comentario truncado, enlace a la ficha).

Prisma mockeado (sin base de datos en tests). No crear E2E.

Definition of Ready

La entrega estará lista para implementarse cuando esta especificación esté aprobada.

Definition of Done

La entrega se considerará finalizada cuando:

- el saludo muestre el nombre del usuario autenticado;
- las cuatro estadísticas reflejen los totales reales;
- los últimos muestren hasta 6 establecimientos recientes;
- el ranking ordene hasta 6 por valoración con medallas;
- los accesos rápidos enlacen a sus destinos;
- `npm run build`, `npm run lint` y `npm test` pasen sin errores.